Southwest Tech

Python Programming Foundations

Week 2 – Decision Logic and Repetition

Day 3: "Combining Logic and Repetition"

1

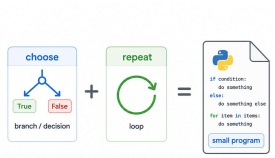
Chose And Repeat

This week, you learned two control-flow tools.

Decisions choose what happens. Loops repeat what happens.

Today, those ideas work together:

- a condition can choose a branch
- a loop can repeat a process
- a condition can also help decide when repetition stops



2

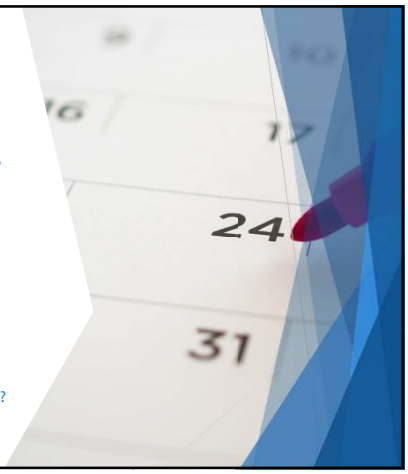
Monday Plus Tuesday

Monday: conditions choose branches.

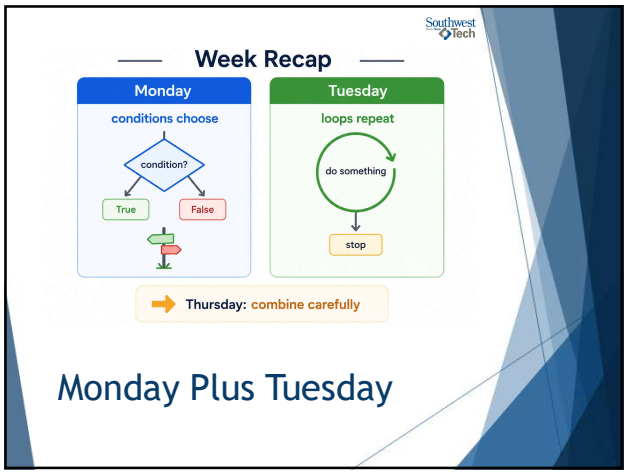
Tuesday: loops repeat work.

Thursday's job is to keep both ideas explainable:

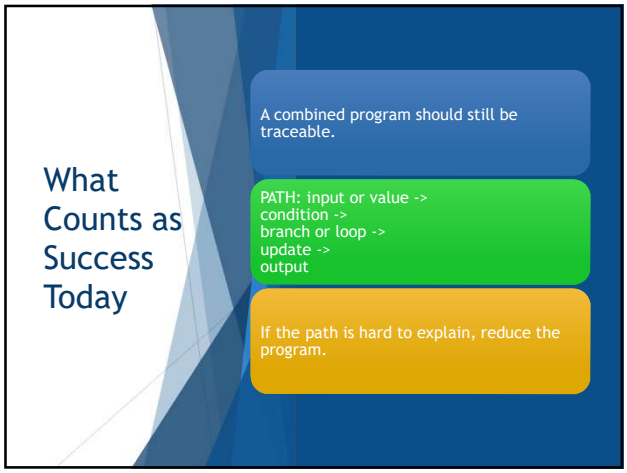
- What condition is checked?
- What repeats?
- What changes?
- What stops the process?



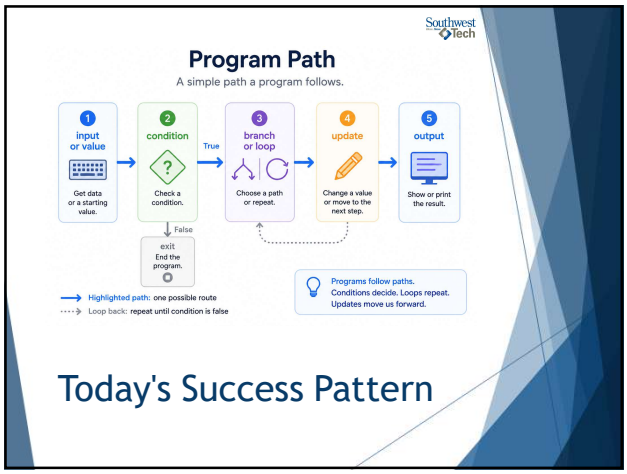
3


Southwest Tech
Week Recap
Monday
conditions choose
condition?
True
False
Tuesday
loops repeat
do something
stop
Thursday: combine carefully
Monday Plus Tuesday

4


What Counts as Success Today
A combined program should still be traceable.
PATH: input or value -> condition -> branch or loop -> update -> output
If the path is hard to explain, reduce the program.

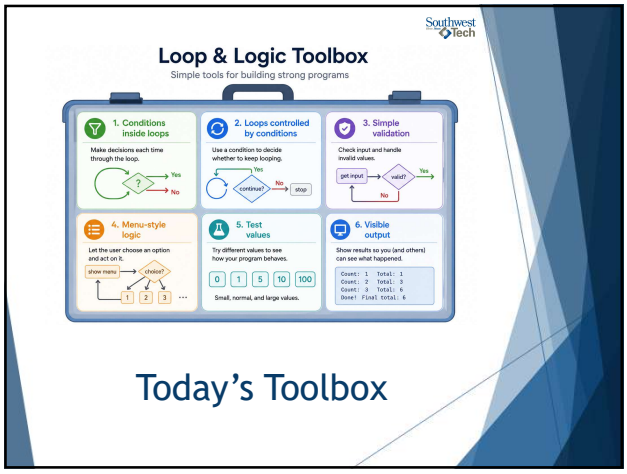
5


Southwest Tech
Program Path
A simple path a program follows.
1 input or value
Get data or a starting value.
2 condition
Check a condition.
True
False
exit
End the program.
3 branch or loop
Choose a path or repeat.
4 update
Change a value or move to the next step.
5 output
Show or print the result.
Highlighted path: one possible route
Loop back: repeat until condition is false
Programs follow paths.
Conditions decide. Loops repeat.
Updates move us forward.
Today's Success Pattern

6


Southwest Tech
What We Will Use Today
conditions inside loops
loops controlled by conditions
simple validation
small menu-style logic
test values and visible output
The goal is controlled combination, not feature count.

7


Southwest Tech
Loop & Logic Toolbox
Simple tools for building strong programs
1. Conditions inside loops
2. Loops controlled by conditions
3. Simple validation
4. Menu-style logic
5. Test values
6. Visible output
Today's Toolbox

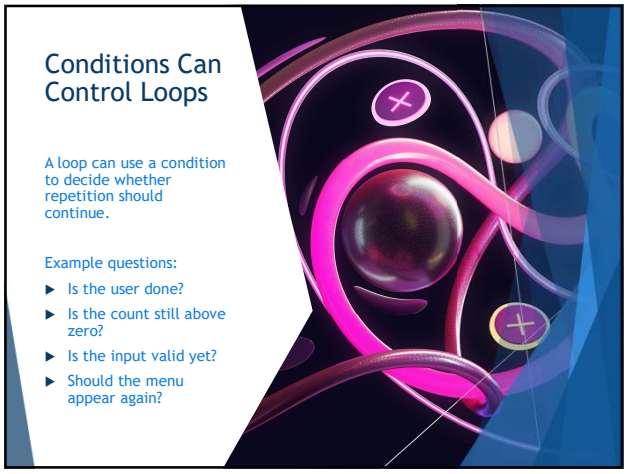
8


Southwest Tech
What We Will "Skip" For Now
large interactive applications
complicated nested logic
Avoid adding features until the main path is clear.
Today, one condition plus one repetition pattern is enough for success.

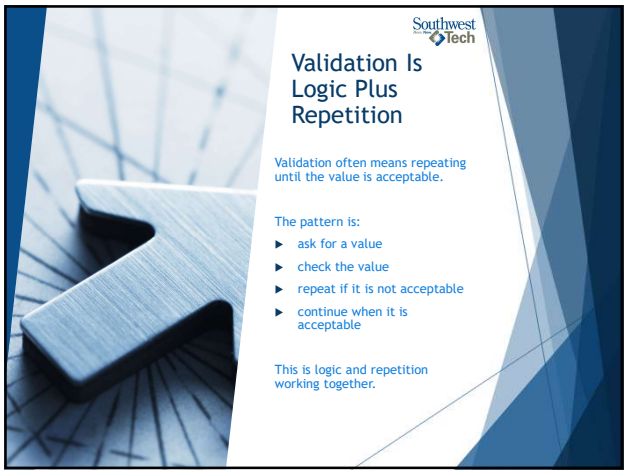
9


Southwest Tech
Parked For Later
These ideas are useful, but they are not today's focus.
SAVE FOR LATER
nested loops
many menu options
multiple features
advanced input handling
Useful ideas, saved until the main path is clear.
"Parked"

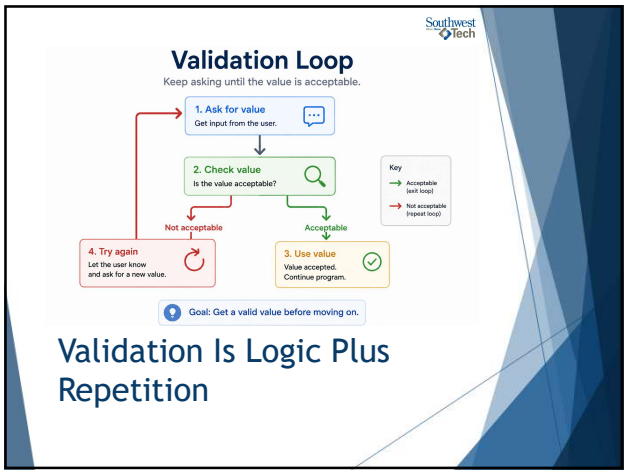
10


Conditions Can Control Loops
A loop can use a condition to decide whether repetition should continue.
Example questions:
Is the user done?
Is the count still above zero?
Is the input valid yet?
Should the menu appear again?

11


Southwest Tech
Validation Is Logic Plus Repetition
Validation often means repeating until the value is acceptable.
The pattern is:
ask for a value
check the value
repeat if it is not acceptable
continue when it is acceptable
This is logic and repetition working together.

12


Southwest Tech
Validation Loop
Keep asking until the value is acceptable.
1. Ask for value
Get input from the user.
2. Check value
Is the value acceptable?
Key
Acceptable (exit loop)
Not acceptable (repeat loop)
Not acceptable
Acceptable
4. Try again
Let the user know and ask for a new value.
3. Use value
Value accepted. Continue program.
Goal: Get a valid value before moving on.
Validation Is Logic Plus Repetition

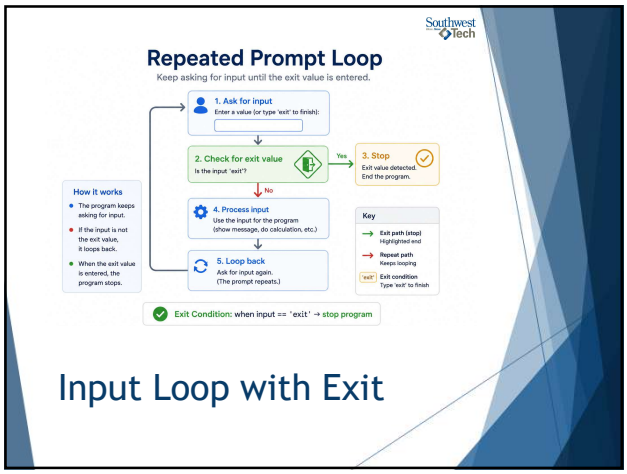
13


Demo #1
"Input Loop w/Exit"

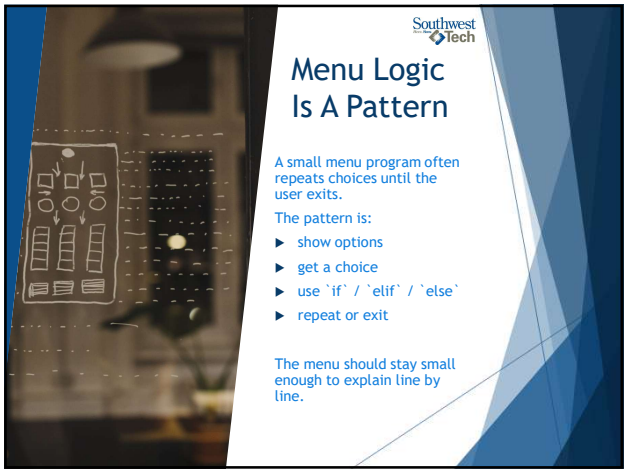
14


Southwest Tech
What to watch for
For each pass, identify:
what input is requested
what condition is checked
whether the loop continues
what value makes the loop stop

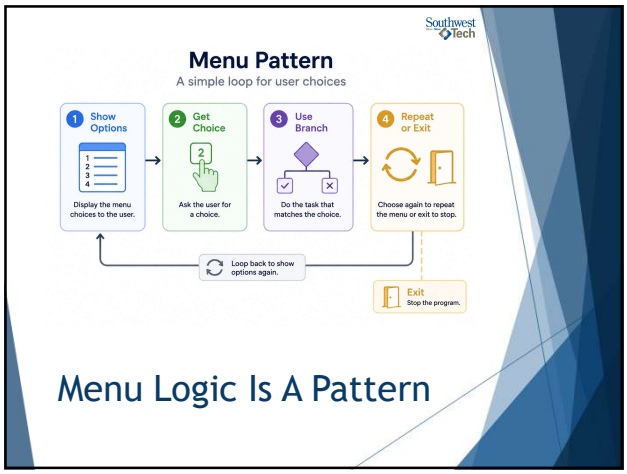
15


Repeated Prompt Loop
Keep asking for input until the exit value is entered.
1. Ask for input
Enter a value (or type 'exit' to finish):
2. Check for exit value
Is the input 'exit'?
Yes
3. Stop
Exit value detected.
End the program.
No
4. Process input
Use the input for the program
(show message, do calculation, etc.)
5. Loop back
Ask for input again.
(The prompt repeats.)
How it works
The program keeps asking for input.
If the input is not the exit value, it loops back.
When the exit value is entered, the program stops.
Key
Exit path (stop)
Highlighted end
Repeat path
Keeps looping
'exit'
Exit condition
Type 'exit' to finish
Exit Condition: when input == 'exit' → stop program
Input Loop with Exit

16


Menu Logic
Is A Pattern
A small menu program often repeats choices until the user exits.
The pattern is:
show options
get a choice
use `if` / `elif` / `else`
repeat or exit
The menu should stay small enough to explain line by line.

17


Menu Pattern
A simple loop for user choices
1 Show Options
Display the menu choices to the user.
2 Get Choice
Ask the user for a choice.
3 Use Branch
Do the task that matches the choice.
4 Repeat or Exit
Choose again to repeat the menu or exit to stop.
Loop back to show options again.
Exit
Stop the program.
Menu Logic Is A Pattern

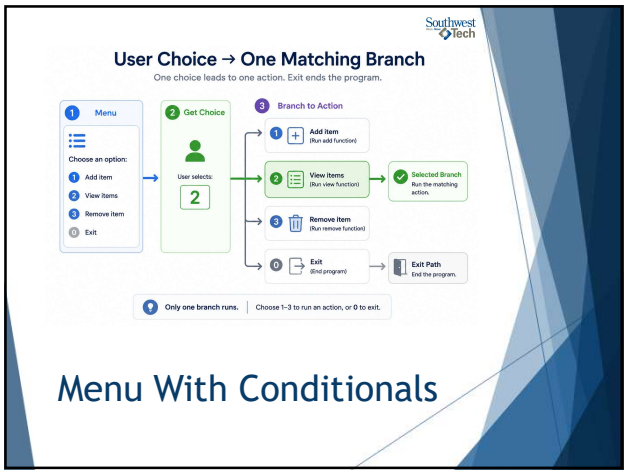
18


Demo #2
"Menu with Conditionals"

19


Southwest Tech
What to watch for
The loop controls repetition. The conditionals control the selected action.
Trace:
the menu display
the user's choice
the matching branch
the exit path

20


Southwest Tech
User Choice → One Matching Branch
One choice leads to one action. Exit ends the program.
1 Menu
Choose an option:
1 Add item
2 View items
3 Remove item
0 Exit
2 Get Choice
User selects:
2
3 Branch to Action
1 Add item
(Run add function)
2 View items
(Run view function)
Selected Branch
Run the matching action.
3 Remove item
(Run remove function)
0 Exit
(End program)
Exit Path
End the program.
Only one branch runs.
Choose 1–3 to run an action, or 0 to exit.
Menu With Conditionals

21

Trace The Whole Path

Combined logic should be traced as a path.

For one run of the program, write or say:

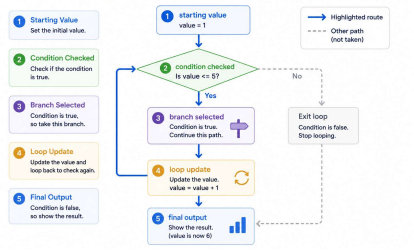
- the starting value or input
- the condition checked
- the branch selected
- the loop update
- the final output

Tracing keeps the program understandable.

22

Combined Logic: One Clear Route

A condition guides the flow, and a loop repeats until the goal is reached.



Trace The Whole Path Slide

23

Test More Than One Path

One successful run is not enough evidence.

For Week 2 work, test:

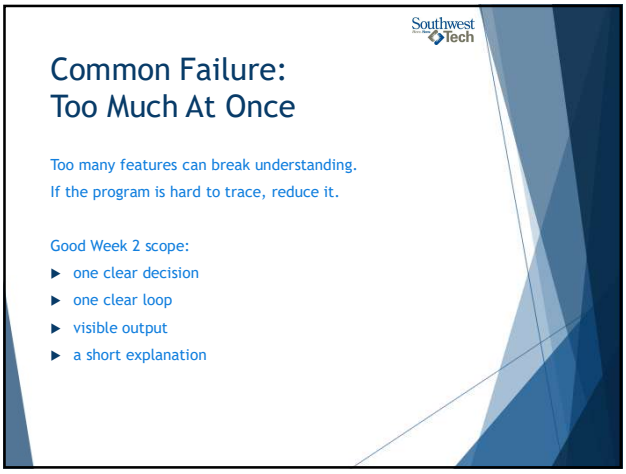
- a value that triggers one branch
- a value that triggers another branch
- a loop that runs more than once
- a value or choice that stops the loop

Testing proves that control flow behaves as intended.

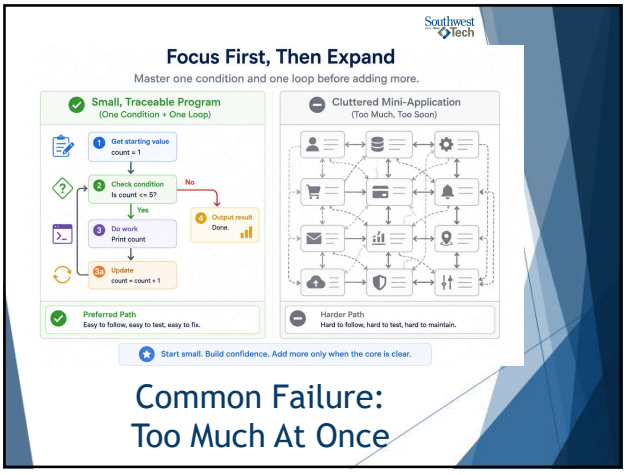
24


Southwest Tech
Quick Logic Checks
Run small tests to build confidence in your logic.
1 Branch A
Condition checked
A
B
Action A
Action B
Test that the program takes Branch A when the condition is true.
Output
Action A ran as expected.
2 Branch B
Condition checked
A
B
Action A
Action B
Test that the program takes Branch B when the condition is false.
Output
Action B ran as expected.
3 Repeat More Than Once
Loop body
Test that the loop runs multiple times before stopping.
Output
Loop ran 3 times as expected.
4 Stop Condition
STOP
Test that the program stops when the stop condition is met.
Output
Stop condition triggered.
All tests passed.
Your logic is working as expected.
Test More Than One Path

25


Southwest Tech
Common Failure:
Too Much At Once
Too many features can break understanding.
If the program is hard to trace, reduce it.
Good Week 2 scope:
one clear decision
one clear loop
visible output
a short explanation

26


Southwest Tech
Focus First, Then Expand
Master one condition and one loop before adding more.
Small, Traceable Program
(One Condition + One Loop)
1 Get starting value
count = 1
2 Check condition
Is count <= 5?
No
Yes
3 Do work
Print count
4 Output result
Done.
3b Update
count = count + 1
Preferred Path
Easy to follow, easy to test, easy to fix.
Cluttered Mini-Application
(Too Much, Too Soon)
Harder Path
Hard to follow, hard to test, hard to maintain.
Start small. Build confidence. Add more only when the core is clear.
Common Failure:
Too Much At Once

27

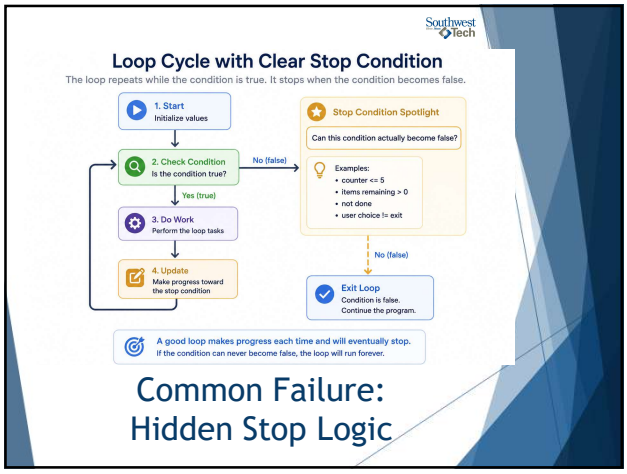
Common Failure: Hidden Stop Logic

A loop should not hide its stopping point.

Before submitting, ask:

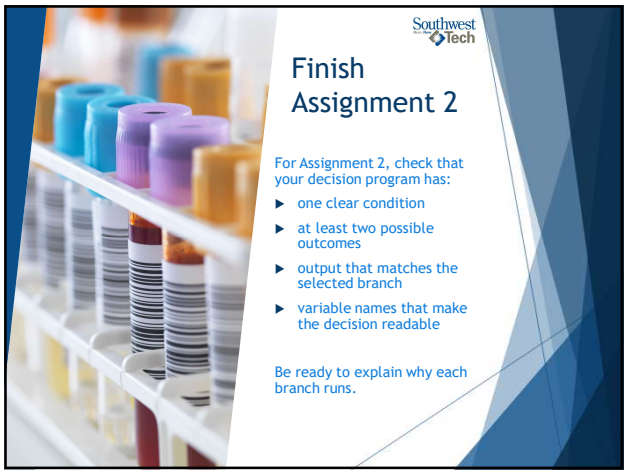
- What makes the loop continue?
- What makes the loop stop?
- Can the stop condition actually happen?
- Can I explain it without guessing?

28



Common Failure: Hidden Stop Logic

29



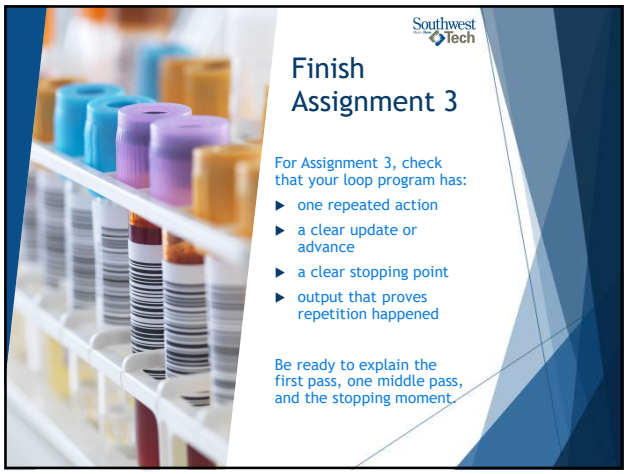
Finish Assignment 2

For Assignment 2, check that your decision program has:

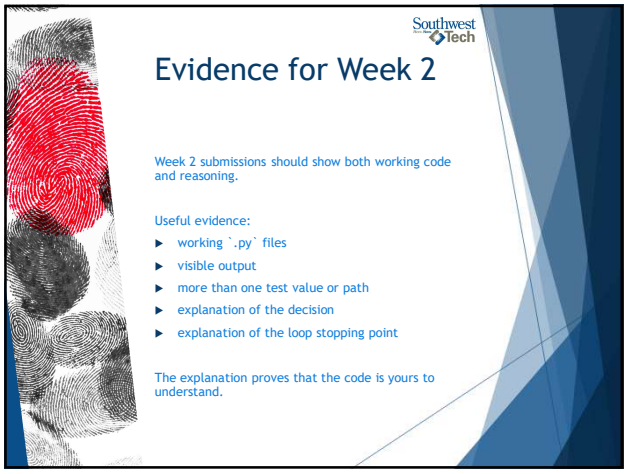
- one clear condition
- at least two possible outcomes
- output that matches the selected branch
- variable names that make the decision readable

Be ready to explain why each branch runs.

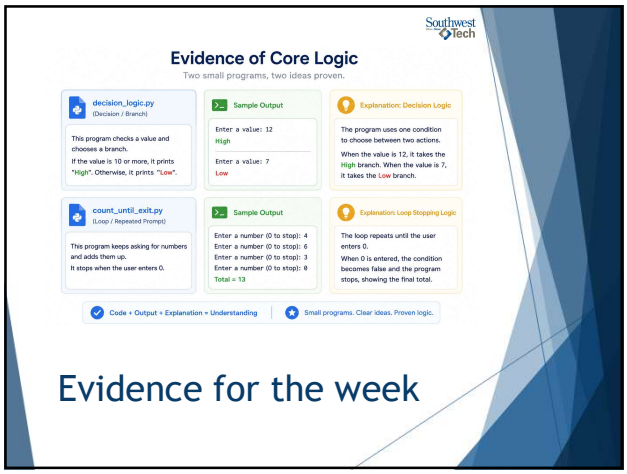
30


Southwest Tech
Finish Assignment 3
For Assignment 3, check that your loop program has:
one repeated action
a clear update or advance
a clear stopping point
output that proves repetition happened
Be ready to explain the first pass, one middle pass, and the stopping moment.

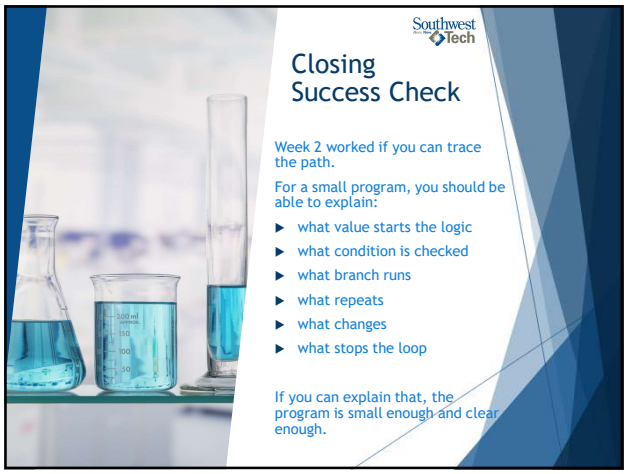
31


Southwest Tech
Evidence for Week 2
Week 2 submissions should show both working code and reasoning.
Useful evidence:
working `.py` files
visible output
more than one test value or path
explanation of the decision
explanation of the loop stopping point
The explanation proves that the code is yours to understand.

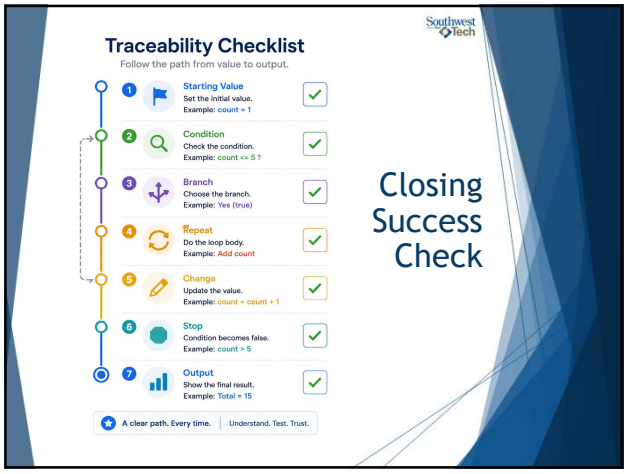
32


Southwest Tech
Evidence of Core Logic
Two small programs, two ideas proven.
decision_logic.py
(Decision / Branch)
This program checks a value and chooses a branch.
If the value is 10 or more, it prints "High". Otherwise, it prints "Low".
Sample Output
Enter a value: 12
High
Enter a value: 7
Low
Explanation: Decision Logic
The program uses one condition to choose between two actions.
When the value is 12, it takes the High branch. When the value is 7, it takes the Low branch.
count_until_exit.py
(Loop / Repeated Prompt)
This program keeps asking for numbers and adds them up.
It stops when the user enters 0.
Sample Output
Enter a number (0 to stop): 4
Enter a number (0 to stop): 6
Enter a number (0 to stop): 3
Enter a number (0 to stop): 0
Total = 13
Explanation: Loop Stopping Logic
The loop repeats until the user enters 0.
When 0 is entered, the condition becomes false and the program stops, showing the final total.
Code + Output + Explanation = Understanding
Small programs. Clear ideas. Proven logic.
Evidence for the week

33


Southwest Tech
Closing Success Check
Week 2 worked if you can trace the path.
For a small program, you should be able to explain:
what value starts the logic
what condition is checked
what branch runs
what repeats
what changes
what stops the loop
If you can explain that, the program is small enough and clear enough.

34


Southwest Tech
Traceability Checklist
Follow the path from value to output.
Starting Value
Set the initial value.
Example: count = 1
Condition
Check the condition.
Example: count <= 5 ?
Branch
Choose the branch.
Example: Yes (true)
Repeat
Do the loop body.
Example: Add count
Change
Update the value.
Example: count = count + 1
Stop
Condition becomes false.
Example: count > 5
Output
Show the final result.
Example: Total = 15
A clear path. Every time.
Understand. Test. Trust.
Closing Success Check

35


Southwest Tech
Thank you!
Have Fun!

36